

NumPy 完整学习笔记（第 1-7 节）

学习者背景：数学本科 + 应统硕士 + 少量 Python 基础

本笔记覆盖：NumPy 入门 → 数组创建 → 属性 → 运算 → 索引切片 → 广播机制 → 实用工具

用途：完整复习材料，包含所有讲解、例子、代码、坑提醒

目录

- 第 1 节：NumPy 是什么？为什么需要它？
- 第 2 节：创建数组（ndarray）
- 第 3 节：数组的核心属性
- 第 4 节：数组的运算（向量化思维）
- 第 5 节：索引与切片
- 第 6 节：广播机制（broadcasting）★★★★
- 第 7 节：实用工具集合
- 核心要点总结

第 1 节：NumPy 是什么？为什么需要它？

一、Python 原生 list 的痛点

数学里你写 $\vec{c} = \vec{a} + \vec{b}$ ，但用 Python list 实现时：

```
a = [1, 2, 3]
b = [10, 20, 30]
c = a + b
print(c)    # [1, 2, 3, 10, 20, 30]    ← 这是拼接，不是逐元素相加！
```

问题： 1. 数学公式只有一行，代码却要多行 2. 慢（Python for 循环效率低） 3. list 不支持矩阵乘法

二、NumPy 的解决方案

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
c = a + b
print(c)    # [11 22 33]    ← 直接就是数学里的向量加法!
```

NumPy 重新定义了运算符，让 `+`、`*`、`@` 在数组上的行为 **和数学一致**。

三、核心思想

NumPy 让你像写数学公式一样写代码。

整个 Python 科学计算生态（Pandas、Scikit-learn、PyTorch、TensorFlow）**底层都是 NumPy**。

四、关键定位

NumPy = 张量数据结构 + 高速运算引擎

- 一维 ndarray = 向量
 - 二维 ndarray = 矩阵
 - 三维及以上 ndarray = 张量（tensor）
-

第 2 节：创建数组（ndarray）

方法 1：从 Python list 创建

```
# 一维（向量）
a = np.array([1, 2, 3])

# 二维（矩阵）
M = np.array([[1, 2, 3],
              [4, 5, 6]])
```

方法 2：特殊矩阵

```
np.zeros((3, 4))    # 3×4 的零矩阵
np.ones((2, 3))     # 全 1 矩阵
np.eye(3)           # 单位矩阵 I_3
```

⚠ **常见坑**：形状要用 **tuple**： `np.zeros((3, 4))` 而不是 `np.zeros(3, 4)`。

方法 3：等差数列

```
np.arange(0, 10, 2)    # [0 2 4 6 8]    起点、终点（不含）、步长
np.linspace(0, 1, 5)   # [0. 0.25 0.5 0.75 1.] 起点、终点（含）、点数
```

函数	边界	第三个参数	默认 dtype
<code>arange</code>	左闭右开	步长	int
<code>linspace</code>	左闭右闭	点数	float

方法 4：随机数

```
# 0-1 之间均匀分布  $U(0, 1)$ 
np.random.rand(3)           # 一维
np.random.rand(3, 4)        # 3×4 矩阵

# 标准正态分布  $N(0, 1)$ 
np.random.randn(3, 4)

# 任意正态分布  $N(\mu, \sigma^2)$ 
np.random.normal(loc=100, scale=15, size=1000)

# 任意均匀分布  $U(\text{low}, \text{high})$ 
np.random.uniform(-1, 1, size=1000)

# 整数:  $[\text{low}, \text{high})$  之间
np.random.randint(0, 10, size=5)

# 随机种子 (科研可复现性必备)
np.random.seed(42)
```

💡 **命名记忆：** `randn` 的 **n** = **n**ormal (正态)。`rand` 没 **n** 就是默认均匀分布。

第 3 节：数组的核心属性

一、4 个核心属性（属性访问，不带括号）

```
M = np.array([[1, 2, 3],
               [4, 5, 6]])

M.shape    # (2, 3)    形状
M.ndim     # 2         维度数
M.dtype    # int64     数据类型
M.size     # 6         总元素个数
```

二、属性 vs 函数

用属性	等价函数写法	推荐
<code>A.shape</code>	<code>np.shape(A)</code>	✓ 属性
<code>A.ndim</code>	<code>np.ndim(A)</code>	✓ 属性
<code>A.size</code>	<code>np.size(A)</code>	✓ 属性
<code>A.dtype</code>	✗ 没有 <code>np.dtype(A)</code>	✓ 唯一写法

三、dtype 的重要性

```
a = np.array([1, 2, 3])           # int64
b = np.array([1.0, 2, 3])         # float64 (混入浮点 → 全部升级)
c = np.array([1, 2, 3], dtype=np.float32)
```

强制 int:

```
np.zeros(5, dtype=int)           # 创建时指定
arr.astype(int)                  # 事后转换
```

四、形状的隐藏陷阱

```
a = np.array([1, 2, 3])          # shape: (3,)      一维向量
b = np.array([[1, 2, 3]])        # shape: (1, 3)   二维行向量
c = np.array([[1], [2], [3]])    # shape: (3, 1)   二维列向量
```

这三种形状不一样，做运算时行为不同！

第 4 节：数组的运算（向量化思维）

一、逐元素运算

```
a + b    # 逐元素加
a - b    # 逐元素减
a * b    # 逐元素乘 (Hadamard 积) ⚠️ 不是矩阵乘!
a / b    # 逐元素除
a ** 2   # 逐元素平方
```

二、矩阵乘法

```
A @ B    # 推荐
np.matmul(A, B)
np.dot(A, B)
```

三、`*` vs `@` 关键区别

符号	含义
<code>*</code>	Hadamard 积（逐元素）
<code>@</code>	矩阵乘法

四、常用矩阵操作

```
A.T          # 转置
A.reshape(3, 2) # 改形状
np.linalg.inv(A) # 求逆
np.linalg.det(A) # 行列式
np.linalg.eig(A) # 特征值/特征向量 (返回 tuple)
np.linalg.norm(x) # L2 范数
np.linalg.solve(A, b) # 解方程组 (推荐! 比 inv 快稳)
```

五、聚合函数与 axis（核心难点）

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])

A.sum()           # 21
A.sum(axis=0)      # [5 7 9]    每列的和
A.sum(axis=1)      # [6 15]     每行的和
```

口诀：axis 是要被消掉的那个维度。

六、向量化思维

```
# Sigmoid 一行实现
1 / (1 + np.exp(-x))

# Softmax 一行实现
np.exp(x) / np.exp(x).sum()
```

第 5 节：索引与切片

一、一维索引

```
a[0]      # 第一个
a[-1]     # 最后一个
a[1:4]    # 切片，左闭右开
a[::-1]   # 反转
```

二、二维索引

```
A[i, j]    # 推荐写法
A[i, :]    # 第 i 行
A[:, j]    # 第 j 列
A[0:2, 1:3] # 子矩阵
```

三、视图 vs 副本

- 切片 → 视图（共享内存）
- 布尔/花式索引 → 副本
- 强制独立副本: `.copy()`

四、布尔索引（核心）

```
a[a > 0]           # 取正数
a[(a > 0) & (a < 5)] # 多条件，必须用 & 加括号
a[a < 0] = 0        # 条件赋值

(y_true == y_pred).mean() # 分类准确率（应用神器）
```

五、花式索引

```
a[[0, 2, 4]]      # 用整数数组当索引
M[:, [0, 2]]      # 取多列
```

六、`np.where` 三元运算

```
np.where(condition, x, y) # 满足条件取 x，否则取 y
```

七、排序

```
np.argsort(a)      # 升序索引
np.argsort(-a)[:k] # top-k 索引
```

第 6 节：广播机制（broadcasting）

★★★★ NumPy 最神奇也最容易出 bug 的特性。这是 PyTorch 张量运算的基础，必须吃透。

一、什么是广播？

之前你已经"无意中"用过广播了：

```
a = np.array([1, 2, 3])
print(a + 10)           # [11 12 13]
```

`a` 是 shape `(3,)`，`10` 是标量。**形状不同也能直接相加**——这就是广播。

更进一步：

```
X = np.array([[1, 2, 3],
               [4, 5, 6]])          # shape (2, 3)
mean = np.array([1, 2, 3])          # shape (3,)

print(X - mean)
# [[0 0 0]
#   [3 3 3]]
```

`(2, 3)` 减 `(3,)` —— 形状不一致，但 NumPy 自动把 `mean` 复制成两行，让形状对齐。

二、广播的核心思想

广播 = 把小数组"虚拟地"复制扩展到大数组的形状，然后逐元素运算。

注意是**虚拟地**——NumPy 不会真的复制内存，只是按规则计算，效率极高。

三、广播的 4 条规则（必背）

NumPy 比较两个数组形状时，**从右往左**逐维比较：

规则 1：如果两个数组维度数不同，**在较小的那个前面补 1**，直到维度数相同。

规则 2：在每一维上，两个数组的大小**要么相等，要么其中一个是 1**。

规则 3：如果某一维满足规则 2，就**沿这一维"复制"较小的那个**，对齐形状。

规则 4：如果某一维不满足规则 2（既不相等也不是 1），**报错**。

四、广播实例分解

例 1: $(2, 3) + (3,) \rightarrow$ 成功

```
X      shape: (2, 3)
mean shape:   (3, )

# 规则 1: mean 前面补 1  $\rightarrow (1, 3)$ 
X      shape: (2, 3)
mean shape: (1, 3)

# 规则 2: 第 0 维 (2 vs 1) 一个是 1  $\checkmark$ ; 第 1 维 (3 vs 3) 相等  $\checkmark$ 
# 规则 3: mean 沿第 0 维复制 2 次  $\rightarrow (2, 3)$ 

最终运算:  $(2, 3) - (2, 3) \rightarrow (2, 3)$ 
```

例 2: $(3, 1) + (1, 4) \rightarrow$ 成功

```
A shape: (3, 1)
B shape: (1, 4)

# 第 0 维: (3 vs 1)  $\rightarrow$  B 复制 3 次
# 第 1 维: (1 vs 4)  $\rightarrow$  A 复制 4 次

最终:  $(3, 4) + (3, 4) \rightarrow (3, 4)$ 
```

```
A = np.array([[1], [2], [3]])    # shape (3, 1)
B = np.array([[10, 20, 30, 40]]) # shape (1, 4)
print(A + B)
# [[11 21 31 41]
#   [12 22 32 42]
#   [13 23 33 43]]
```

💡 **数学含义**: 这就是**外积** (outer sum / outer product 的加法版本)。

例 3: `(2, 3) + (2,)` → ❌ 报错

```
X shape: (2, 3)
y shape: (2,)

# 规则 1: y 前面补 1 → (1, 2)
X shape: (2, 3)
y shape: (1, 2)

# 第 1 维: (3 vs 2) → 不相等也不是 1 → 报错!
```

💡 **解决办法**: 把 `y` reshape 成 `(2, 1)` 就能广播:

```
X = np.array([[1, 2, 3],
               [4, 5, 6]])
y = np.array([10, 20])

# 错: X + y          报错
# 对: X + y[:, None] # 等价于 y.reshape(2, 1)
```

五、`np.newaxis` 和 `None`: 扩展维度的神器

`np.newaxis` (等价于 `None`) 可以在指定位置插入一个 `size=1` 的维度:

```
a = np.array([1, 2, 3])      # shape (3,)

# 变成行向量
a[np.newaxis, :]             # shape (1, 3)
a[None, :]                   # 等价

# 变成列向量
a[:, np.newaxis]             # shape (3, 1)
a[:, None]                   # 等价
```

🔗 **生产代码常用写法**: `a[:, None]` 比 `a.reshape(-1, 1)` 短, 效果一样。

六、广播的经典应用

应用 1：矩阵每行减去均值（Z-score 标准化）

```
X = np.array([[1, 2, 3],
              [4, 5, 6]])

mean = X.mean(axis=0)    # shape (3,) 每列的均值
X_centered = X - mean    # 广播: (2,3) - (3,) → 自动按列减
```

应用 2：矩阵每行乘以一个权重向量

```
X = np.random.randn(100, 5)    # 100 个样本, 5 个特征
weights = np.array([2, 1, 0.5, 1, 3]) # 每个特征的权重

X_weighted = X * weights        # (100, 5) * (5,) → 自动广播
```

应用 3：向量化欧氏距离矩阵 ★

给定 n 个 d 维点，算两两距离矩阵 D ， $D[i,j] = ||x_i - x_j||$ 。

普通写法（双重循环）：

```
n = X.shape[0]
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.linalg.norm(X[i] - X[j])
```

广播写法（一行）：

```
diff = X[:, None, :] - X[None, :, :]    # shape (n, n, d)
D = np.sqrt((diff ** 2).sum(axis=-1))    # shape (n, n)
```

拆解：- $X[:, \text{None}, :]$ shape $(n, 1, d)$ —— 每个点变成“列” - $X[\text{None}, :, :]$ shape $(1, n, d)$ —— 每个点变成“行” - 相减广播成 (n, n, d) —— $[i, j]$ 位置就是 $X[i] - X[j]$ - 平方求和开方 → 距离

💡 这是**生产代码的标准模式**。在做 K-Means、KNN、推荐系统时几乎天天用。

应用 4：外积 (outer product)

数学外积 $\vec{a} \otimes \vec{b}$ ，结果是 $a_i b_j$ 的矩阵：

```
a = np.array([1, 2, 3])      # shape (3,)
b = np.array([10, 20, 30])  # shape (3,)

# 方法 1: 广播
outer1 = a[:, None] * b[None, :]  # shape (3, 3)

# 方法 2: np.outer
outer2 = np.outer(a, b)           # 等价

# [[10 20 30]
#  [20 40 60]
#  [30 60 90]]
```

应用 5：归一化每个样本（按行标准化）

```
# 让每行的范数 = 1 (K-Means 中常用)
X_normalized = X / np.linalg.norm(X, axis=1, keepdims=True)
```

注意 `keepdims=True`：保持 `(n, 1)` 形状，**让广播能对齐**。如果不加 `keepdims`，结果是 `(n,)`，广播会出错。

七、`keepdims` 参数（广播的好朋友）

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])

A.sum(axis=1)           # shape (2,)      [6 15]
A.sum(axis=1, keepdims=True) # shape (2, 1)  [[6] [15]]
```

为什么需要？**保留维度让后续广播能对齐**：

```
# 行归一化 (每行除以该行的和)
A / A.sum(axis=1, keepdims=True)    # ✓ (2,3) / (2,1) → 广播成功
A / A.sum(axis=1)                    # ✗ (2,3) / (2,) 形状不匹配
```

📌 记住口诀：做完聚合还要广播 → 加 `keepdims=True`。

八、广播的形状判断技巧

实战中怎么判断两个数组能不能广播？

1. 把两个 shape 右对齐写出来
2. 看每一列：要么相等，要么至少一个是 1
3. 都满足 → 能广播；任一不满足 → 报错

能广播的例子：

(2, 3, 4)
(3, 4) ✓ 补成 (1, 3, 4)，每列相等或为 1

(5, 1, 4)
(1, 6, 1) ✓ 结果是 (5, 6, 4)

不能广播的例子：

(2, 3)
(3, 2) ✗ 第 1 维 3≠2 且都不是 1

九、广播报错的 debug 技巧

```
# ValueError: operands could not be broadcast together with shapes (2,3)
```

3 步定位： 1. print 两个数组的 shape 2. 右对齐对比每一维 3. 找出不匹配的那一维，决定 reshape 还是用 `np.newaxis`

第 7 节：实用工具集合

这一节是工具补全。日常用 NumPy 90% 离不开这些函数。

一、数组拼接

np.concatenate：通用拼接

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

np.concatenate([A, B], axis=0) # 纵向拼接 (4, 2)
np.concatenate([A, B], axis=1) # 横向拼接 (2, 4)
```

简写版

```
np.vstack([A, B]) # 等价 axis=0 (vertical stack)
np.hstack([A, B]) # 等价 axis=1 (horizontal stack)
```

np.stack：增加新维度

```
A = np.array([1, 2, 3])
B = np.array([4, 5, 6])

np.stack([A, B]) # shape (2, 3), 新增第 0 维
np.stack([A, B], axis=1) # shape (3, 2)
```

💡 **concatenate** 和 **stack** 的区别： - **concatenate**：在**已有维度**上拼接（不增维） - **stack**：**创建新维度**

二、数组分割

```
arr = np.arange(12).reshape(3, 4)

# 分成 2 份 (沿 axis=0, 需要能整除)
np.split(arr, 2, axis=0)
# [array([[0, 1, 2, 3]]), array([[4, 5, 6, 7]])]

# 不能整除时用 array_split (任意分)
np.array_split(arr, 3, axis=0)

# 按索引位置切分
np.split(arr, [1, 3], axis=1)    # 在列索引 1 和 3 处切分
```

三、维度操作

```
a = np.array([1, 2, 3])

# 增加维度
a[:, None]          # shape (3, 1)
a[None, :]          # shape (1, 3)
np.expand_dims(a, 0) # shape (1, 3), 等价 None 写法

# 删除大小为 1 的维度
b = np.array([[[[1, 2, 3]]]]) # shape (1, 1, 3)
b.squeeze()                 # shape (3,)
b.squeeze(axis=0)           # shape (1, 3) 只挤掉指定维

# 重排维度 (转置的推广)
A = np.random.rand(2, 3, 4)
A.transpose(1, 0, 2)        # shape (3, 2, 4)
```

四、复制与重复

```
a = np.array([1, 2, 3])

# 重复整个数组
np.tile(a, 3)          # [1 2 3 1 2 3 1 2 3]
np.tile(a, (2, 3))     # 在两个方向上分别重复

# 重复每个元素
np.repeat(a, 3)        # [1 1 1 2 2 2 3 3 3]
np.repeat(a, [1, 2, 3]) # [1 2 2 3 3 3] 每个元素的次数不同
```

五、数据 IO（保存与加载）

NumPy 二进制格式（推荐）

```
# 单个数组
np.save('data.npy', arr)
arr_loaded = np.load('data.npy')

# 多个数组打包
np.savez('data.npz', X=X, y=y, mean=mean)
data = np.load('data.npz')
X = data['X']
y = data['y']

# 压缩存储（适合稀疏数据）
np.savez_compressed('data.npz', X=X)
```

 **优势：**完整保留 dtype 和 shape，加载快，比 csv 体积小很多。

文本格式

```
np.savetxt('data.csv', arr, delimiter=',', fmt='%.4f')
arr = np.loadtxt('data.csv', delimiter=',')
```

 **实践建议：** - 中间结果存 `.npy` / `.npz`（快、省） - 给别人看的存 `.csv`（通用）

六、统计常用函数

np.unique：去重 + 计数

```
a = np.array([1, 2, 2, 3, 3, 3, 4])

np.unique(a)                # [1 2 3 4]    去重
np.unique(a, return_counts=True) # ([1 2 3 4], [1 2 3 1]) 带计数
```

应用：统计分类标签分布

```
labels = np.array([0, 1, 1, 2, 0, 2, 1, 1])
classes, counts = np.unique(labels, return_counts=True)
print(dict(zip(classes, counts))) # {0: 2, 1: 4, 2: 2}
```

np.bincount：非负整数计数（更快）

```
a = np.array([0, 1, 1, 2, 2, 2])
np.bincount(a) # [1 2 3] 表示 0 出现 1 次, 1 出现 2 次, 2 出现 3 次
```

 **应用**：计算分类问题的类别分布、做 one-hot 编码、统计直方图。

np.histogram：直方图

```
data = np.random.randn(1000)
counts, bin_edges = np.histogram(data, bins=20)
# counts: 每个 bin 里的点数
# bin_edges: 21 个边界值

# 配合 matplotlib 画
import matplotlib.pyplot as plt
plt.hist(data, bins=20)
```

七、累计计算

```
a = np.array([1, 2, 3, 4, 5])

np.cumsum(a)      # [1 3 6 10 15]    累计求和
np.cumprod(a)     # [1 2 6 24 120]   累计乘积
```

应用：计算累计收益率、大数定律演示。

八、条件查找

`np.where` 的另一种用法

只传一个参数时，返回**满足条件的索引**：

```
a = np.array([10, 20, 30, 40, 50])
np.where(a > 25)      # (array([2, 3, 4]),)  返回索引
```

二维矩阵：

```
A = np.array([[1, 5, 3],
               [4, 2, 6]])
rows, cols = np.where(A > 3)
# rows = [0, 1, 1]
# cols = [1, 0, 2]
# 表示位置 (0,1), (1,0), (1,2) 的值大于 3
```

`np.any` 和 `np.all`

```
a = np.array([True, False, True])

np.any(a)      # True    至少有一个 True
np.all(a)      # False   是否全部 True

# 配合条件
np.any(arr > 100)      # 数组中是否有任何元素 > 100
np.all(arr >= 0)       # 是否全部非负
```

九、最大/最小元素索引（多维）

```
A = np.array([[1, 5, 3],
               [4, 2, 6]])

A.argmax()           # 5           扁平后的索引
np.unravel_index(A.argmax(), A.shape) # (1, 2)   还原成多维索引

A.argmax(axis=0)      # [1 0 1]    每列最大值的行索引
A.argmax(axis=1)      # [1 2]      每行最大值的列索引
```

十、`np.clip`：截断

```
a = np.array([-2, 0, 3, 5, 10])
np.clip(a, 0, 5)           # [0 0 3 5 5]   把值限制在 [0, 5] 之间
```

应用：异常值处理、梯度裁剪。

十一、`np.sort` vs `np.argsort`

```
a = np.array([3, 1, 4, 1, 5, 9, 2])

np.sort(a)           # [1 1 2 3 4 5 9]   排序后的值
np.argsort(a)        # [1 3 6 0 2 4 5]   升序索引

# 矩阵每行/每列排序
np.sort(M, axis=1)
np.argsort(M, axis=0)
```

十二、生成网格点（画图常用）

```
x = np.linspace(-2, 2, 5)
y = np.linspace(-2, 2, 5)

X, Y = np.meshgrid(x, y)
# X 和 Y 都是 (5, 5) 的矩阵
# 用 X[i, j], Y[i, j] 可以表示一个 5x5 网格上的所有点
```

应用：画等高线、3D 曲面、决策边界。

十三、综合应用：手写 K-Means（一个完整算法）

把前面所有知识点串起来：

```
def kmeans(X, k, max_iter=100):
    n, d = X.shape
    # 随机初始化中心
    centers = X[np.random.choice(n, k, replace=False)]

    for _ in range(max_iter):
        # 1. 用广播算距离矩阵 (n, k)
        distances = np.linalg.norm(
            X[:, None, :] - centers[None, :, :], axis=-1
        )
        # 2. 每个点分配到最近中心
        labels = distances.argmin(axis=1)
        # 3. 更新中心
        new_centers = np.array([X[labels == j].mean(axis=0) for j in range(k)])
        # 4. 收敛检查
        if np.allclose(centers, new_centers):
            break
        centers = new_centers

    return labels, centers
```

这个 10 行函数用到了：广播、距离计算、`argmin`、布尔索引、聚合（`mean`）、收敛判断（`allclose`）——几乎覆盖了 NumPy 全部核心功能。

核心要点总结

必须牢记的 9 件事

1. NumPy = 张量数据结构 + 高速运算引擎
2. 形状是一切的基础：`(3,)`、`(1,3)`、`(3,1)` 不同
3. `*` 是逐元素乘，`@` 才是矩阵乘
4. `axis` 是被消掉的维度：`axis=0` 消掉行
5. 能用向量化就别用 `for` 循环
6. 切片返回视图，需要副本就 `.copy()`
7. 布尔索引超强大：`(y_true == y_pred).mean()` 是灵魂

8. 广播 = 小数组虚拟扩展到大数组形状
9. 聚合后还要广播 → 加 `keepdims=True`

常踩的坑速查表

错误写法	正确写法	原因
<code>np.zeros(3, 4)</code>	<code>np.zeros((3, 4))</code>	形状要用 tuple
<code>A * B</code> (想做矩阵乘)	<code>A @ B</code>	<code>*</code> 是 Hadamard 积
<code>(3)</code>	<code>(3,)</code>	单元素 tuple 必须加逗号
<code>axis=0</code> (想要每行的和)	<code>axis=1</code>	axis 是被消掉的维度
<code>a and b</code> (数组逻辑与)	<code>a & b</code>	<code>and</code> 不能用于数组
<code>(a > 0) & (a < 5)</code> 漏括号	必须每个条件加括号	<code>&</code> 优先级高
<code>np.linalg.inv(A) @ b</code>	<code>np.linalg.solve(A, b)</code>	inv 慢且不稳
<code>eigvecs[i]</code> (取特征向量)	<code>eigvecs[:, i]</code>	特征向量是按列存
<code>A / A.sum(axis=1)</code>	<code>A / A.sum(axis=1, keepdims=True)</code>	广播形状不对
<code>(2, 3) + (2,)</code>	reshape 成 <code>(2, 1)</code> 再加	广播规则不满足

工具速查表

```
# === 创建 ===
np.array(list)          np.zeros(shape)
np.ones(shape)          np.eye(n)
np.arange(start, stop, step) np.linspace(start, stop, num)
np.random.rand/randn/uniform/normal/randint/seed

# === 属性 ===
.shape .ndim .dtype .size

# === 运算 ===
+ - * /                # 逐元素
@                      # 矩阵乘
.T                    # 转置
.reshape(...)         # 改形状

# === 线代 ===
np.linalg.inv/det/eig/norm/solve

# === 聚合 (注意 axis 和 keepdims) ===
.sum/mean/std/var/max/min/argmax/argmin

# === 数学函数 (逐元素) ===
np.exp/log/sqrt/sin/cos/abs
np.maximum/minimum

# === 索引 ===
A[i, j] A[i, :] A[:, j] A[i1:i2, j1:j2]
A[A > 0] A[[0, 2, 4]]
np.where(cond, x, y) np.argsort(a)

# === 广播工具 ===
a[:, None] a[None, :] # 等价 np.newaxis
np.expand_dims np.squeeze
keepdims=True

# === 拼接分割 ===
np.concatenate / vstack / hstack / stack
np.split / array_split

# === 复制重复 ===
np.tile np.repeat

# === IO ===
np.save / load / savez / loadtxt / savetxt
```

```
# === 统计 ===  
np.unique np.bincount np.histogram  
np.cumsum np.cumprod  
np.any np.all np.clip  
  
# === 类型 ===  
.astype(int) .copy()
```

常用数学/ML 公式速查（一行实现）

```
# Sigmoid  $\sigma(x) = 1/(1+e^{(-x)})$ 
1 / (1 + np.exp(-x))

# Softmax
np.exp(x) / np.exp(x).sum()

# ReLU
np.maximum(0, x)

# L2 范数
np.linalg.norm(x)

# 余弦相似度
(a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Z-score 标准化
(X - X.mean(axis=0)) / X.std(axis=0)

# 行归一化（用广播）
X / np.linalg.norm(X, axis=1, keepdims=True)

# 分类准确率
(y_true == y_pred).mean()

# MSE
((y_true - y_pred) ** 2).mean()

# 最小二乘线性回归  $\beta$ 
np.linalg.solve(X.T @ X, X.T @ y)

# 协方差矩阵
np.cov(X, rowvar=False)

# 距离矩阵（广播版）
np.linalg.norm(X[:, None] - X[None, :], axis=-1)

# 外积
a[:, None] * b[None, :]
```

学完 NumPy 你能做什么

- ☒ 独立完成 Kaggle 入门赛的数据预处理
- ☒ 看懂任何 NumPy 代码（包括开源项目和论文复现）
- ☒ 无缝过渡到 Pandas（Pandas 底层就是 NumPy）
- ☒ 无缝过渡到 PyTorch（语法 90% 相似）
- ☒ 手撕经典 ML 算法（K-Means、KNN、线性回归、PCA）

下一步

NumPy 部分到此结束。下一步是 **Pandas（数据处理王者）** —— 应统人会爱死它，很多概念直接对应 R 的 dplyr。